

# Báo cáo Bài tập 2: Quy Hoạch Động

## Vũ Tiến Đạt - 23000111

### 1 Bài toán 1: Tìm chuỗi con chung dài nhất (LCS)

#### 1.1 Đặt bài toán

Cho hai chuỗi  $X$  có độ dài  $m$  và  $Y$  có độ dài  $n$ . Hãy tìm độ dài chuỗi con chung của  $X$  và  $Y$  có độ dài lớn nhất. Biết chuỗi con thu được khi xóa một số ký tự (hoặc không xóa) mà vẫn giữ nguyên thứ tự.

#### 1.2 Thiết kế và phân tích thuật toán

**Xây dựng trạng thái:** Gọi  $dp[i][j]$  là độ dài chuỗi con chung dài nhất của  $i$  ký tự đầu tiên trong  $X$  ( $X[1 \dots i]$ ) và  $j$  ký tự đầu tiên trong  $Y$  ( $Y[1 \dots j]$ ).

**Bài toán cơ sở:**  $dp[i][0] = 0$  và  $dp[0][j] = 0$  với mọi  $0 \leq i \leq m, 0 \leq j \leq n$ .

**Công thức truy hồi:**

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{nếu } X[i] = Y[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{nếu } X[i] \neq Y[j] \end{cases} \quad (1)$$

**Độ phức tạp:**

- **Thời gian:**  $\mathcal{O}(m \cdot n)$  do cần duyệt qua bảng kích thước  $m \times n$ .
- **Không gian:**  $\mathcal{O}(m \cdot n)$  để lưu trữ bảng phương án  $dp$ .

## 1.3 Mã giả (Pseudocode)

---

**Algorithm 1** Tìm độ dài chuỗi con chung dài nhất (LCS)

---

**Require:** Hai chuỗi  $X$  (độ dài  $m$ ),  $Y$  (độ dài  $n$ )

**Ensure:** Độ dài chuỗi con chung lớn nhất

```
1: Khởi tạo mảng  $dp[0 \dots m][0 \dots n]$  với toàn số 0
2: for  $i \leftarrow 1$  to  $m$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:     if  $X[i] == Y[j]$  then
5:        $dp[i][j] \leftarrow dp[i-1][j-1] + 1$ 
6:     else
7:        $dp[i][j] \leftarrow \max(dp[i-1][j], dp[i][j-1])$ 
8:     end if
9:   end for
10: end for
11: return  $dp[m][n]$ 
```

---

## 2 Bài toán 2: Thêm ký tự để tạo chuỗi đối xứng

### 2.1 Đặt bài toán

Cho một chuỗi  $S$  độ dài  $n$ . Hãy tìm số ký tự ít nhất cần thêm vào  $S$  để  $S$  trở thành chuỗi đối xứng (palindrome).

### 2.2 Thiết kế và phân tích thuật toán

**Nhận xét:** Những ký tự đã tạo thành chuỗi con đối xứng thì không cần thêm. Số ký tự cần thêm bằng tổng số ký tự của chuỗi ban đầu trừ đi độ dài *Chuỗi con đối xứng dài nhất (LPS)* của  $S$ . Mặt khác, độ dài LPS của  $S$  chính là độ dài LCS của chuỗi  $S$  và chuỗi đảo ngược của nó  $S_{rev}$ .

**Công thức:**

$$\text{Kết quả} = n - \text{LCS}(S, S_{rev}) \quad (2)$$

**Độ phức tạp:**

- **Thời gian:**  $\mathcal{O}(n^2)$  do sử dụng thuật toán LCS cho 2 chuỗi độ dài  $n$ .
- **Không gian:**  $\mathcal{O}(n^2)$ .

## 2.3 Mã giả (Pseudocode)

---

**Algorithm 2** Số phép thêm tối thiểu tạo Palindrome

---

**Require:** Xâu  $S$  độ dài  $n$

**Ensure:** Số ký tự ít nhất cần thêm

```
1:  $S_{rev} \leftarrow \text{Reverse}(S)$ 
2: Khởi tạo mảng  $dp[0 \dots n][0 \dots n]$  với toàn số 0
3: for  $i \leftarrow 1$  to  $n$  do
4:   for  $j \leftarrow 1$  to  $n$  do
5:     if  $S[i] == S_{rev}[j]$  then
6:        $dp[i][j] \leftarrow dp[i-1][j-1] + 1$ 
7:     else
8:        $dp[i][j] \leftarrow \max(dp[i-1][j], dp[i][j-1])$ 
9:     end if
10:  end for
11: end for
12:  $LPS\_length \leftarrow dp[n][n]$ 
13: return  $n - LPS\_length$ 
```

---

## 3 Bài toán 3: Xây cầu (Building Bridges)

### 3.1 Đặt bài toán

Hai nước A và B ở hai bên bờ sông (A bờ Bắc có  $M$  thành phố, B bờ Nam có  $N$  thành phố). Có danh sách các cặp thành phố kết nghĩa có thể xây cầu. Yêu cầu tìm cách xây số lượng cầu lớn nhất sao cho mỗi thành phố chỉ nối tối đa 1 cầu và các cây cầu không cắt nhau.

### 3.2 Thiết kế và phân tích thuật toán

**Xây dựng trạng thái:** Bài toán tương đương với việc tìm tập hợp các đường nối sao cho không có 2 đường  $(i_1, j_1)$  và  $(i_2, j_2)$  nào thỏa mãn  $i_1 < i_2$  mà  $j_1 > j_2$ . Cấu trúc này giống bài toán LCS. Gọi  $C[i][j] = 1$  nếu thành phố Bắc  $i$  kết nghĩa với Nam  $j$ , ngược lại bằng 0. Gọi  $dp[i][j]$  là số lượng cầu tối đa xây được khi chỉ xét  $i$  thành phố bờ Bắc và  $j$  thành phố bờ Nam.

**Công thức truy hồi:**

$$dp[i][j] = \begin{cases} \max(dp[i-1][j], dp[i][j-1], dp[i-1][j-1] + 1) & \text{nếu } C[i][j] = 1 \\ \max(dp[i-1][j], dp[i][j-1]) & \text{nếu } C[i][j] = 0 \end{cases} \quad (3)$$

**Độ phức tạp:**

- Thời gian:  $\mathcal{O}(M \cdot N)$ .
- Không gian:  $\mathcal{O}(M \cdot N)$ .

### 3.3 Mã giả (Pseudocode)

---

**Algorithm 3** Xây cầu tối đa

---

**Require:** Số lượng thành phố  $M, N$  và ma trận kết nghĩa  $C[M][N]$

**Ensure:** Số cây cầu tối đa

```
1: Khởi tạo mảng  $dp[0 \dots M][0 \dots N]$  với toàn số 0
2: for  $i \leftarrow 1$  to  $M$  do
3:   for  $j \leftarrow 1$  to  $N$  do
4:      $dp[i][j] \leftarrow \max(dp[i-1][j], dp[i][j-1])$ 
5:     if  $C[i][j] == 1$  then
6:        $dp[i][j] \leftarrow \max(dp[i][j], dp[i-1][j-1] + 1)$ 
7:     end if
8:   end for
9: end for
10: return  $dp[M][N]$ 
```

---

## 4 Bài toán 4: Chia nhóm có tích tổng lớn nhất

### 4.1 Đặt bài toán

Cho  $n$  số nguyên. Hãy chia chúng thành 2 nhóm sao cho tích của tổng 2 nhóm là lớn nhất.

### 4.2 Thiết kế và phân tích thuật toán

**Nhận xét:** Gọi  $Sum$  là tổng của tất cả các phần tử. Giả sử tổng nhóm 1 là  $S_1$ , tổng nhóm 2 là  $S_2 = Sum - S_1$ . Tích  $P = S_1 \cdot (Sum - S_1)$  đạt giá trị lớn nhất khi  $S_1$  càng gần  $Sum/2$  càng tốt. Ta quy về bài toán Cái túi (Subset Sum): Tìm tập con có tổng  $\leq \lfloor Sum/2 \rfloor$  sao cho tổng đó lớn nhất có thể.

**Xây dựng trạng thái:** Gọi  $dp[j]$  là biến boolean lưu trữ xem có tồn tại một nhóm có tổng bằng  $j$  hay không. Khởi tạo  $dp[0] = \text{True}$ , các giá trị khác là  $\text{False}$ .

**Công thức truy hồi:** Khi xét phần tử thứ  $i$ , cập nhật ngược từ  $\lfloor Sum/2 \rfloor$  về  $A[i]$ :

$$dp[j] = dp[j] \vee dp[j - A[i]] \quad (4)$$

**Độ phức tạp:**

- Thời gian:  $\mathcal{O}(n \cdot Sum)$ .
- Không gian:  $\mathcal{O}(Sum)$ .

### 4.3 Mã giả (Pseudocode)

---

**Algorithm 4** Chia 2 nhóm có tích lớn nhất

---

**Require:** Mảng  $A$  có  $n$  phần tử

**Ensure:** Tích lớn nhất của 2 nhóm

```
1:  $Sum \leftarrow \sum_{i=1}^n A[i]$ 
2:  $Target \leftarrow \lfloor Sum/2 \rfloor$ 
3: Khởi tạo mảng boolean  $dp[0 \dots Target]$  với  $dp[0] = \text{True}$ , còn lại  $\text{False}$ 
4: for  $i \leftarrow 1$  to  $n$  do
5:   for  $j \leftarrow Target$  downto  $A[i]$  do
6:     if  $dp[j - A[i]] == \text{True}$  then
7:        $dp[j] \leftarrow \text{True}$ 
8:     end if
9:   end for
10: end for
11:  $S_1 \leftarrow 0$ 
12: for  $j \leftarrow Target$  downto  $0$  do
13:   if  $dp[j] == \text{True}$  then
14:      $S_1 \leftarrow j$ 
15:     break
16:   end if
17: end for
18:  $S_2 \leftarrow Sum - S_1$ 
19: return  $S_1 \times S_2$ 
```

---

## Luyện tập 1

**Đề bài:** Chuyển đổi công thức truy hồi sau thành code

$$T(0) = T(1) = 2$$

$$T(n) = \sum_{i=1}^{n-1} 2 \cdot T(i) \cdot T(i-1) \quad , n > 1$$

**Giải quyết:** Đây là cách tiếp cận đệ quy nguyên bản dựa trên công thức toán học.

---

**Algorithm 5** Tính  $T(n)$  bằng đệ quy (Brute Force)

---

```
1: function CALCULATE_T(n)
2:   if  $n == 0$  or  $n == 1$  then
3:     return 2
4:   end if
5:   sum  $\leftarrow$  0
6:   for  $i \leftarrow 1$  to  $n - 1$  do
7:     sum  $\leftarrow$  sum +  $2 \times \text{CALCULATE\_T}(i) \times \text{CALCULATE\_T}(i-1)$ 
8:   end for
9:   return sum
10: end function
```

---

## Luyện tập 2

**Đề bài:** Có thể cải thiện giải pháp của Luyện tập 1 bằng quy hoạch động không? Vì sao?

**Trả lời: Có thể.**

**Vì sao?** Thuật toán đệ quy ở Luyện tập 1 gặp phải vấn đề *bài toán con gối nhau* (*Overlapping Subproblems*). Để tính  $T(n)$ , nó gọi lại  $T(n-1), T(n-2), \dots$  rất nhiều lần, dẫn đến độ phức tạp thời gian theo hàm mũ  $\mathcal{O}(2^n)$ . Bằng cách sử dụng Quy hoạch động (lưu kết quả vào mảng  $dp$  từ dưới lên), ta chỉ cần tính mỗi  $T(i)$  đúng một lần. Độ phức tạp thời gian giảm xuống còn  $\mathcal{O}(n^2)$  và không gian là  $\mathcal{O}(n)$ .

## Luyện tập 3

**Đề bài:** Chúng ta có thể cải thiện hơn nữa về độ phức tạp của Luyện tập 2 không? Vì sao?

**Trả lời: Có thể.** Chúng ta có thể giảm độ phức tạp thời gian từ  $\mathcal{O}(n^2)$  xuống  $\mathcal{O}(n)$  và không gian từ  $\mathcal{O}(n)$  xuống  $\mathcal{O}(1)$ .

**Vì sao?** Bằng cách biến đổi toán học để khử dấu tổng  $\sum$ :

$$T(n) = \sum_{i=1}^{n-1} 2 \cdot T(i) \cdot T(i-1)$$
$$T(n-1) = \sum_{i=1}^{n-2} 2 \cdot T(i) \cdot T(i-1)$$

Lấy  $T(n) - T(n - 1)$ , ta được:

$$T(n) - T(n - 1) = 2 \cdot T(n - 1) \cdot T(n - 2) \implies T(n) = T(n - 1) \cdot [1 + 2 \cdot T(n - 2)]$$

Công thức truy hồi mới chỉ phụ thuộc vào 2 giá trị liền trước nó, không cần vòng lặp lồng nhau tính tổng.

## Luyện tập 4

**Đề bài:** (*Maximum Value Contiguous Subsequence*) Cho một mảng có  $n$  phần tử, hãy đưa ra một thuật toán để tìm dãy con liên tiếp  $A(i) \dots A(j)$  mà tổng các phần tử của nó là lớn nhất. Ví dụ:  $\{-2, 11, -4, 13, -5, 2\}$  với output là: 20 hoặc  $\{1, -3, 4, -2, -1, 6\}$  với output là: 7.

**Giải quyết:** Thuật toán cơ bản nhất (Brute Force) là duyệt qua mọi cặp chỉ số  $(i, j)$  và tính tổng. Thuật toán này có độ phức tạp  $\mathcal{O}(n^2)$  (nếu cộng dồn tối ưu) hoặc  $\mathcal{O}(n^3)$ .

---

**Algorithm 6** Tìm dãy con tổng lớn nhất ( $\mathcal{O}(n^2)$ )

---

```
1: function MAXSUBARRAY_NAIVE( $A, n$ )
2:    $max\_sum \leftarrow -\infty$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:      $current\_sum \leftarrow 0$ 
5:     for  $j \leftarrow i$  to  $n - 1$  do
6:        $current\_sum \leftarrow current\_sum + A[j]$ 
7:       if  $current\_sum > max\_sum$  then
8:          $max\_sum \leftarrow current\_sum$ 
9:       end if
10:    end for
11:  end for
12:  return  $max\_sum$ 
13: end function
```

---

## Luyện tập 5

**Đề bài:** Có thể cải thiện độ phức tạp của Luyện tập 4 hay không? Vì sao?

**Trả lời:** Có thể.

**Vì sao?** Chúng ta có thể sử dụng phương pháp *Chia để trị* (*Divide and Conquer*). Chia mảng làm hai nửa, dãy con lớn nhất có thể nằm trọn ở nửa trái, trọn ở nửa phải, hoặc vắt ngang qua điểm giữa. Kỹ thuật này giúp giảm độ phức tạp thời gian từ  $\mathcal{O}(n^2)$  xuống  $\mathcal{O}(n \log n)$ .

## Luyện tập 6

**Đề bài:** Có thể cải thiện độ phức tạp của Luyện tập 4 bằng cách sử dụng Quy hoạch động hay không? Vì sao?

**Trả lời:** Có thể.

**Vì sao?** Quy hoạch động có thể đưa bài toán về thời gian  $\mathcal{O}(n)$ . Gọi  $dp[i]$  là tổng lớn

nhất của dãy con liên tiếp **kết thúc tại vị trí  $i$** . Tại phần tử  $A[i]$ , ta có hai lựa chọn: nối  $A[i]$  vào dãy con kết thúc tại  $i - 1$ , hoặc bắt đầu một dãy con mới chỉ có  $A[i]$ .

$$dp[i] = \max(A[i], dp[i - 1] + A[i])$$

Kết quả cuối cùng là  $\max(dp[0 \dots n - 1])$ . Tuy nhiên, cách này cần mảng  $dp$  tốn  $\mathcal{O}(n)$  bộ nhớ.

## Luyện tập 7

**Đề bài:** Có cách nào khác để giải quyết Luyện tập 4 hay không? Vì sao? *Gợi ý: Bằng cách sử dụng Quy hoạch động không có bộ nhớ (Dynamic Programming without memory).*

**Trả lời:** Có. Đó chính là **Thuật toán Kadane**.

**Vì sao?** Nhìn vào công thức Quy hoạch động ở Luyện tập 6:  $dp[i] = \max(A[i], dp[i - 1] + A[i])$ , ta thấy  $dp[i]$  chỉ phụ thuộc vào đúng phần tử ngay trước nó là  $dp[i - 1]$ . Do đó, ta không cần lưu trữ toàn bộ mảng  $dp$  làm gì, mà chỉ cần một biến **current\_max** để lưu trạng thái trước đó, và biến **global\_max** để cập nhật kết quả tốt nhất. Thuật toán đạt thời gian  $\mathcal{O}(n)$  và **không gian**  $\mathcal{O}(1)$ .

---

**Algorithm 7** Thuật toán Kadane (DP without memory)

---

```
1: function KADANE( $A, n$ )
2:    $current\_max \leftarrow A[0]$ 
3:    $global\_max \leftarrow A[0]$ 
4:   for  $i \leftarrow 1$  to  $n - 1$  do
5:      $current\_max \leftarrow \max(A[i], current\_max + A[i])$ 
6:     if  $current\_max > global\_max$  then
7:        $global\_max \leftarrow current\_max$ 
8:     end if
9:   end for
10:  return  $global\_max$ 
11: end function
```

---